5.1 Tree-based methods

Vladimir Buskin

English Language and Linguistics, Catholic University of Eichstätt-Ingolstadt

ORCID: 0009-0005-5824-1012

Last updated: 15 October 2026, 20:09

This handout introduces decision trees and random forests, two powerful non-parametric methods based on recursive partitioning. It covers classification and regression trees (CART), pruning, conditional inference trees, random forests, variable importance measures and partial dependence plots, using data on Russian T/V pronouns and lexical decision times.

Table of contents

Recommended reading . . . 1
Introduction . . . 2
Decision trees . . . 3
- Classification trees . . . 4
- Pruning . . . 5
- Conditional inference trees . . . 7
- Regression trees . . . 7

Random forests . . . 9
- Regression forest . . . 9
- Classification forest . . . 10
- Variable importance . . . 11
- Visualising random forest models . . . 15

References . . . 18

Recommended reading

For linguists:

Levshina (2015): Chapter 14

Levshina (2020)

Gries (2021): Chapter 7

General:

James et al. (2021): Chapter 8

Hastie, Tibshirani, and Friedman (2017): Chapters 9.2 & 15

Introduction

Decision trees and random forests are very popular **non-parametric** methods. As such, "they do not make explicit assumptions about the functional form of f" (James et al. 2021: 23).

In this unit, we will cover the conceptual basics of these methods as well as their implementation in R using the `tv` data from Levshina (2020) in addition to the `ELP` data from the unit on Linear Regression. The libraries we will need are listed below:

```
# Load libraries
library("tidyverse")
library("readxl")
library("tree") # for CART
library("randomForest") # for traditional random forests
library("party") # for conditional inference trees and forests
library("pdp") # for partial dependence plots

## Reaction time data
ELP <- read_xlsx("ELP.xlsx")

ELP$POS <- as.factor(ELP$POS)

## Levshina's (2020) data set on T/V forms in Russian
tv <- read.csv("Levshina_2020_tv_data.csv", sep = ",", header = TRUE,
 ↪  stringsAsFactors = TRUE)
```

In the `tv` data frame, our target variable will be T/V `Form` with the two outcomes *ty* (Russian 2.p.sg., informal) and *vy* (Russian 2.p.pl., polite).

```
str(tv)
```

```
'data.frame':   228 obs. of  17 variables:
 $ Form      : Factor w/ 2 levels "ty","vy": 2 2 1 1 1 2 1 1 1 1 ...
 $ Film      : Factor w/ 9 levels "Avatar","BlackSwan",..: 1 1 1 1 1 1 1 1 1 1 ...
 $ Rel_Age   : Factor w/ 4 levels "Older","Other",..: 4 3 3 3 3 1 4 3 3 3 ...
 $ Rel_Sex   : Factor w/ 4 levels "F_F","F_M","M_F",..: 4 3 2 2 3 4 4 3 2 4 ...
 $ Rel_Power : Factor w/ 3 levels "Equal","Greater",..: 1 3 1 1 1 2 3 1 1 3 ...
 $ Rel_Circle: Factor w/ 6 levels "Acq","Fam","Fri",..: 5 6 6 6 6 6 6 5 5 6 ...
```

```
 $ S_Class   : Factor w/ 4 levels "Lower","Middle",..: 2 2 2 2 2 2 2 2 3 2 ...
 $ H_Class   : Factor w/ 4 levels "Lower","Middle",..: 2 2 2 2 2 2 2 3 2 2 ...
 $ S_Age     : Factor w/ 5 levels "Child","Middle",..: 2 2 5 5 5 5 2 5 5 2 ...
 $ H_Age     : Factor w/ 5 levels "Child","Middle",..: 5 2 5 5 5 2 5 5 5 5 ...
 $ Rel_Class : Factor w/ 4 levels "Equal","Higher",..: 3 1 1 1 1 2 3 4 4 3 ...
 $ Before68  : Factor w/ 2 levels "No","Yes": 1 1 1 1 1 1 1 1 1 1 ...
 $ Others    : Factor w/ 2 levels "No","Yes": 2 2 2 1 1 1 1 1 1 2 ...
 $ Office    : Factor w/ 2 levels "No","Yes": 2 2 2 2 2 2 2 1 1 1 ...
 $ S_Sex     : Factor w/ 2 levels "F","M": 2 2 1 1 2 2 2 2 1 2 ...
 $ H_Sex     : Factor w/ 2 levels "F","M": 2 1 2 2 1 2 2 1 2 2 ...
 $ Place     : Factor w/ 5 levels "Europe","Fictive",..: 2 2 2 2 2 2 2 2 2 2 ...
```

```
head(tv)
```

```
  Form   Film Rel_Age Rel_Sex Rel_Power Rel_Circle S_Class H_Class  S_Age
1   vy Avatar Younger     M_M     Equal        Str  Middle  Middle Middle
2   vy Avatar    Same     M_F      Less       Work  Middle  Middle Middle
3   ty Avatar    Same     F_M     Equal       Work  Middle  Middle  Young
4   ty Avatar    Same     F_M     Equal       Work  Middle  Middle  Young
5   ty Avatar    Same     M_F     Equal       Work  Middle  Middle  Young
6   vy Avatar   Older     M_M   Greater       Work  Middle  Middle  Young
   H_Age Rel_Class Before68 Others Office S_Sex H_Sex   Place
1  Young     Lower       No    Yes    Yes     M     M Fictive
2 Middle     Equal       No    Yes    Yes     M     F Fictive
3  Young     Equal       No    Yes    Yes     F     M Fictive
4  Young     Equal       No     No    Yes     F     M Fictive
5  Young     Equal       No     No    Yes     M     F Fictive
6 Middle    Higher       No     No    Yes     M     M Fictive
```

Decision trees

Core concepts:

- **Segmenting the feature space**: "[T]he feature space (i.e., the space spanned by all predictor variables) is recursively partitioned into a set of rectangular areas" (Strobl, Malley, and Tutz 2009: 325).
- **Impurity reduction**: These simplified prediction areas should consist of mostly homogeneous (i.e., 'pure' rather than 'mixed') observations.
- **Tree construction**: The 'decisions' made when partitioning the training data can be visualised using tree structures. The internal nodes of a tree represent the variables used for splitting, the branches represent decision rules, and the leaf (or terminal) nodes indicate the final outcome (e.g., a prediction).

- **CART**: The most influential algorithm for constructing decision trees is CART (Classification and Regression Trees), developed by Breiman (1984).

Classification trees

If we are dealing with a **categorical response variable**, the `tree()` function can be used to fit a classification tree in accordance with Breiman's CART algorithm. For illustration, consider the `tv` data frame. We will model the choice of the pronoun `Form` based on the speaker's and hearer's social circle (`Rel_Circle`) and their difference in social class (`Rel_Class`).

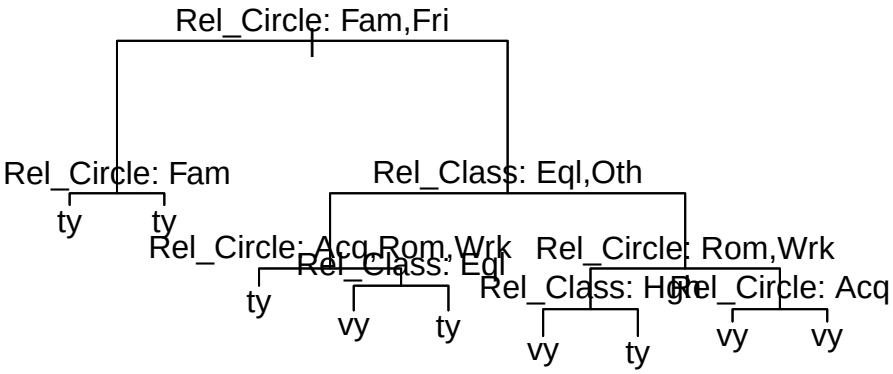
```
# Supply model formula
tree.tv <- tree(Form ~ Rel_Circle + Rel_Class, data = tv)

# View tree statistics
summary(tree.tv)
```

```
Classification tree:
tree(formula = Form ~ Rel_Circle + Rel_Class, data = tv)
Number of terminal nodes:  9
Residual mean deviance:  0.974 = 213.3 / 219
Misclassification error rate: 0.2412 = 55 / 228
```

```
# Visualisation
plot(tree.tv)

text(tree.tv, pretty = 3)
```



An important problem that arises during tree construction is that of **split selection**. When should the tree split a node into two further nodes and when not? Furthermore, when should the tree **stop** the splitting process entirely? In this respect, CART relies on the principle of **impurity reduction**: "The fundamental idea is to select each split of a subset so that

the data in each of the descendent subsets are 'purer' than the data in the parent subset" (Breiman 1984: 23). A measure for node purity is the **Gini index** G, which is defined as

$$G = \sum_{k=1}^{K} \hat{p}_{mk}(1 - \hat{p}_{mk}),$$

where $\hat{p}_{mk}$ measures the proportion of observations of a response level k in the mth prediction area of the training data set. Values close to 0 are indicative of high node purity, meaning that most observations belong to the same class (e.g., `Form = ty`). A closely related measure is the **deviance** (or cross-entropy), which yields very similar results (James et al. 2021: Chapter 8.1.2). In fact, `tree()` uses the deviance by default; the Gini index can be requested with `split = "gini"`.

Once a node is sufficiently pure or contains too few observations, it is no longer split and becomes a **terminal node**, which returns the tree's class prediction (see `?tree.control` for the default thresholds).

Pruning

Note that some splits in the tree above lead to two terminal nodes with the **same** prediction. Such splits make the nodes purer, i.e., the proportions of *ty* and *vy* become more extreme, but they do not change the predicted class.

More generally, large trees tend to **overfit** the training data: They capture idiosyncrasies of the sample that will not generalise to new data. Instead of stopping the splitting process early, CART therefore grows a large tree first and then **prunes** it back to a smaller subtree (Breiman 1984; James et al. 2021: Chapter 8.1.1). The optimal tree size can be determined via cross-validation, which estimates how many misclassifications each subtree would produce on unseen data. Since cross-validation splits the data randomly, we set a seed for reproducibility.

```
# Set random number generator for reproducibility
set.seed(123)

# Cross-validate trees of different sizes
cv.tv <- cv.tree(tree.tv, FUN = prune.misclass)

# size = number of terminal nodes; dev = number of cross-validated
 ↪  misclassifications
cv.tv
```

```
$size
[1] 9 7 5 4 2 1

$dev
```

```
[1]  64  67  72  75  82 107

$k
[1] -Inf  0.0  1.5  3.0  7.0 33.0

$method
[1] "misclass"

attr(,"class")
[1] "prune"         "tree.sequence"
```

```
# Choose the smallest tree with the lowest error
best_size <- min(cv.tv$size[cv.tv$dev == min(cv.tv$dev)])

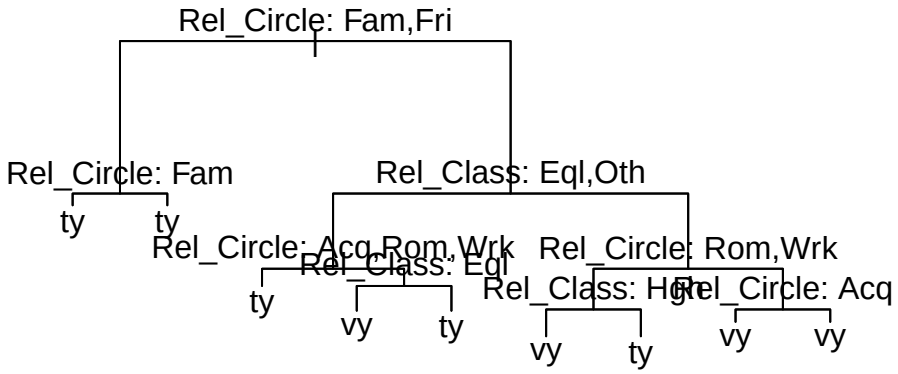
# Prune the tree
pruned.tv <- prune.misclass(tree.tv, best = best_size)

summary(pruned.tv)
```

```
Classification tree:
tree(formula = Form ~ Rel_Circle + Rel_Class, data = tv)
Number of terminal nodes:  9
Residual mean deviance:  0.974 = 213.3 / 219
Misclassification error rate: 0.2412 = 55 / 228
```

```
plot(pruned.tv)

text(pruned.tv, pretty = 3)
```

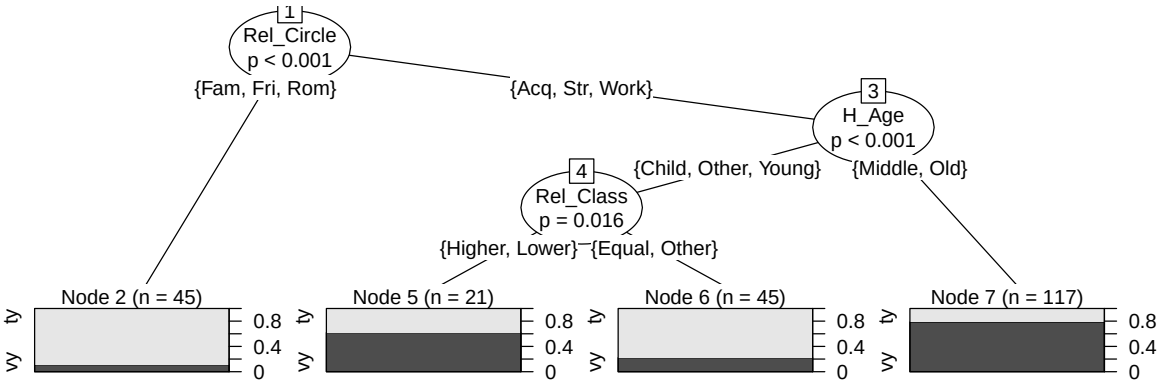


Conditional inference trees

Other tree algorithms rely on different splitting criteria. **Conditional inference trees**, for instance, use statistical tests: At each node, the association between the response and each predictor is tested. If none of the associations is significant (after correcting for multiple testing), the node becomes a terminal node. Otherwise, the data are split on the predictor with the strongest association, i.e., the smallest p-value (Levshina 2020; for more details, see B. Greenwell 2022: 122). Since the tests determine when to stop, no pruning is necessary.

```
# Fitting a conditional inference tree
ctree.tv <- ctree(Form ~ ., data = tv) # dot . means 'include all
 ↪  predictors'

plot(ctree.tv)
```



Regression trees

Regression trees are used for **continuous response variables**. Instead of providing class predictions, they return the mean value of observations in a given prediction area. The algorithm now strives to **minimise the residual sum of squares** (RSS). Consider the regression tree for reaction times depending on word length, frequency and part of speech:

```
# CART tree
tree.rt <- tree(RT ~ Length + Freq + POS, data = ELP)

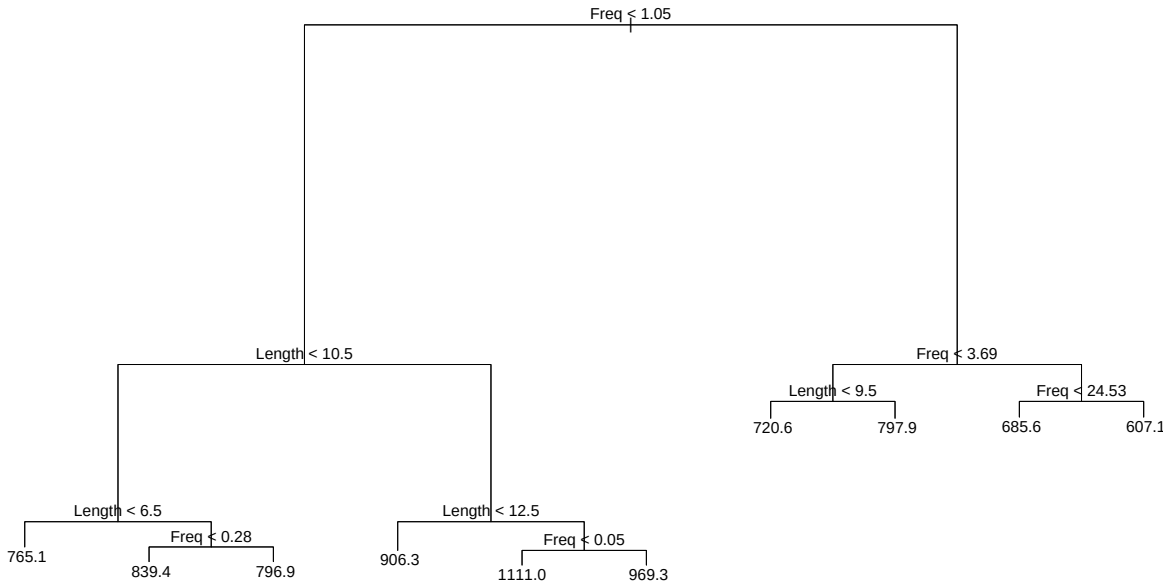
summary(tree.rt)
```

```
Regression tree:
tree(formula = RT ~ Length + Freq + POS, data = ELP)
Variables actually used in tree construction:
[1] "Freq"   "Length"
```

```
Number of terminal nodes:  10
Residual mean deviance:  8629 = 7507000 / 870
Distribution of residuals:
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
-220.10  -59.09  -11.17    0.00   49.66  397.40
```

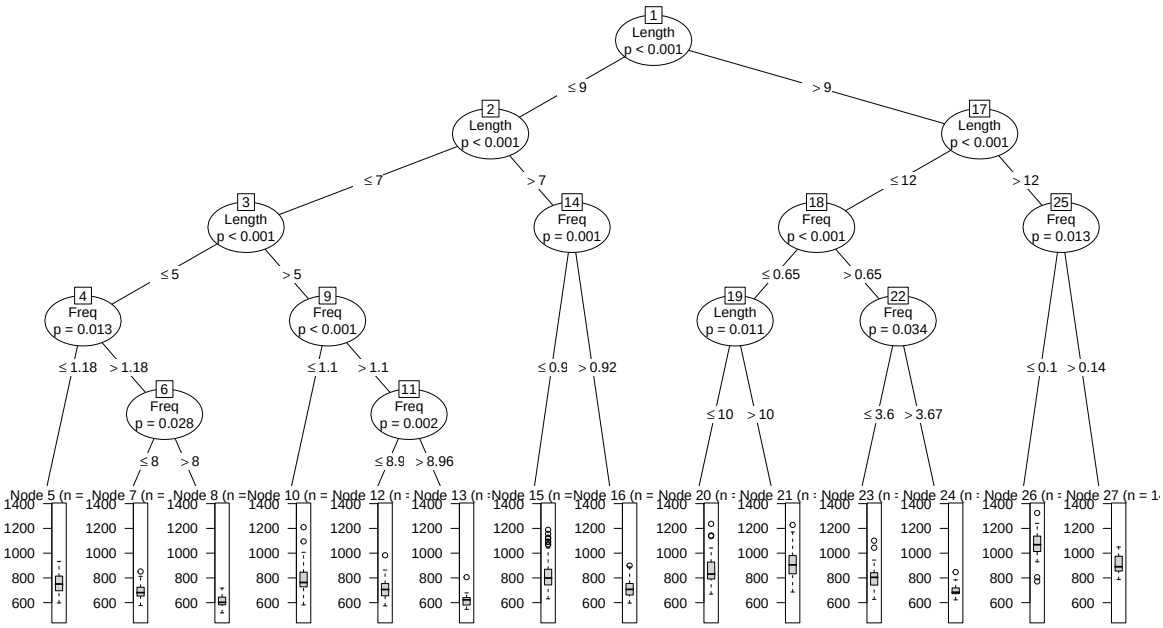
```r
plot(tree.rt)

text(tree.rt, pretty = 0)
```




```r
# Conditional inference tree
ctree.rt <- ctree(RT ~ Length + Freq + POS, data = ELP)

plot(ctree.rt)
```



Random forests

Random forests (Breiman 2001) belong to the class of **ensemble methods** because they combine simpler models (e.g., individual decision trees) into a more complex and possibly more accurate model. As part of the RF algorithm, a great number of decision trees is trained on bootstrapped samples of the training data.

So far, random forests are essentially identical with **Bagging** (= bootstrap aggregation); however, an important additional characteristic of the RF algorithm is that only **a random subset of the predictors** is taken into consideration at each split. According to Strobl et al. (2009: 332), the resulting variability in tree structure is advantageous: "By combining the prediction of such a diverse set of trees, ensemble methods utilize the fact that classification trees are unstable, but, on average, produce the right prediction".

Conditional random forests, which are fitted with `cforest()`, combine conditional inference trees instead of CART trees. With the settings in `cforest_unbiased()`, each tree is fitted to a subsample drawn **without** replacement, which avoids biased variable importance scores (Strobl et al. 2007).

Since the bootstrap samples and the predictor subsets are drawn randomly, we set a seed before fitting the forests.

Regression forest

For **regression** tasks, random forests return the average prediction of all trees in the ensemble.

```
# For reproducibility
set.seed(123)

# For regression
rt.rf.reg <- randomForest(RT ~ Length + Freq + POS, data = ELP,
                          mtry = 1, # default for regression: number of
                           ↪  predictors / 3, rounded down
                          ntree = 500, # number of trees
                          importance = TRUE) # also compute permutation
                           ↪  importance

rt.rf.reg
```

```
Call:
 randomForest(formula = RT ~ Length + Freq + POS, data = ELP,      mtry = 1, ntree = 500,
               Type of random forest: regression
                     Number of trees: 500
No. of variables tried at each split: 1

          Mean of squared residuals: 8992.825
                    % Var explained: 43.51
```

```
# Conditional random forest
rt.crf.reg <- cforest(RT ~ Length + Freq + POS, data = ELP,
                      controls = cforest_unbiased(ntree = 500, mtry = 1))
```

Classification forest

For **classification**, all trees cast a vote for one of the response classes, and the majority vote determines the prediction.

Since each tree is fitted to a bootstrap sample, roughly a third (about 37%) of the observations is not used for fitting that tree. These **out-of-bag (OOB)** observations can serve as an internal test data set: Each observation is predicted only by those trees for which it was out of bag. The **OOB estimate of the error rate** is the proportion of misclassified OOB predictions. For regression forests, the OOB error is the mean of squared residuals shown in the output above.

```
# For reproducibility
set.seed(123)

# For classification
```

```
tv.rf.class <- randomForest(Form ~ ., data = tv,
                            mtry = 4, # default for classification: square
                             ↪  root of the number of predictors, rounded
                             ↪  down
                            ntree = 500,
                            importance = TRUE)

tv.rf.class
```

```
Call:
 randomForest(formula = Form ~ ., data = tv, mtry = 4, ntree = 500,      importance = TRUE
               Type of random forest: classification
                     Number of trees: 500
No. of variables tried at each split: 4

        OOB estimate of  error rate: 19.3%
Confusion matrix:
   ty vy class.error
ty 87 21   0.1944444
vy 23 97   0.1916667
```

```
# Conditional random forest
tv.crf.class <- cforest(Form ~ ., data = tv,
                        controls = cforest_unbiased(ntree = 500, mtry = 4))

tv.crf.class
```

```
     Random Forest using Conditional Inference Trees

Number of trees:  500

Response:  Form
Inputs:  Film, Rel_Age, Rel_Sex, Rel_Power, Rel_Circle, S_Class, H_Class, S_Age, H_Age, Re
Number of observations:  228
```

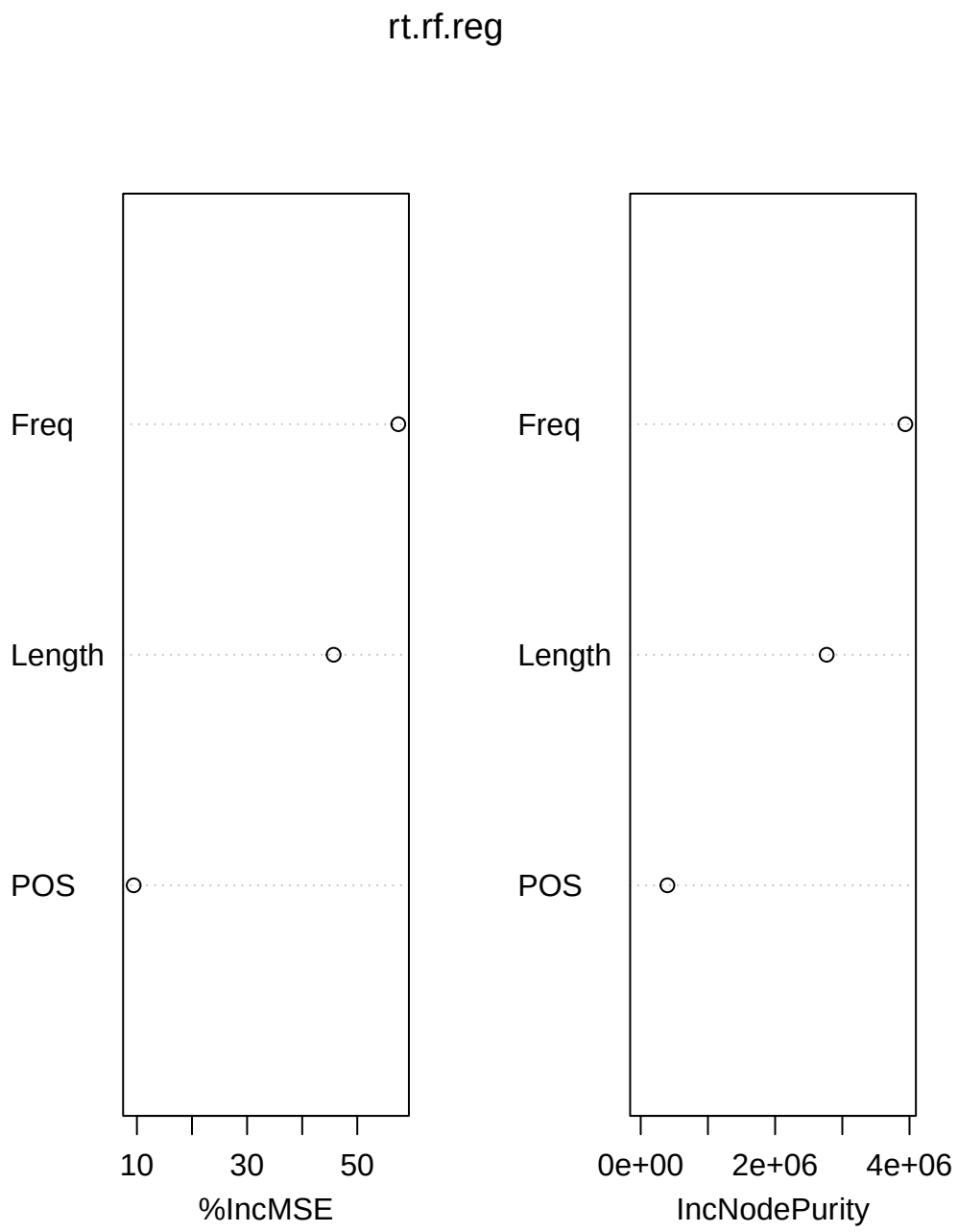
Variable importance

Random forests allow users to assess whether or not certain predictors are useful for the model. The impurity measures from above can be re-used to identify those variables that have led to the greatest reduction in impurity, i.e., the Gini index for classification

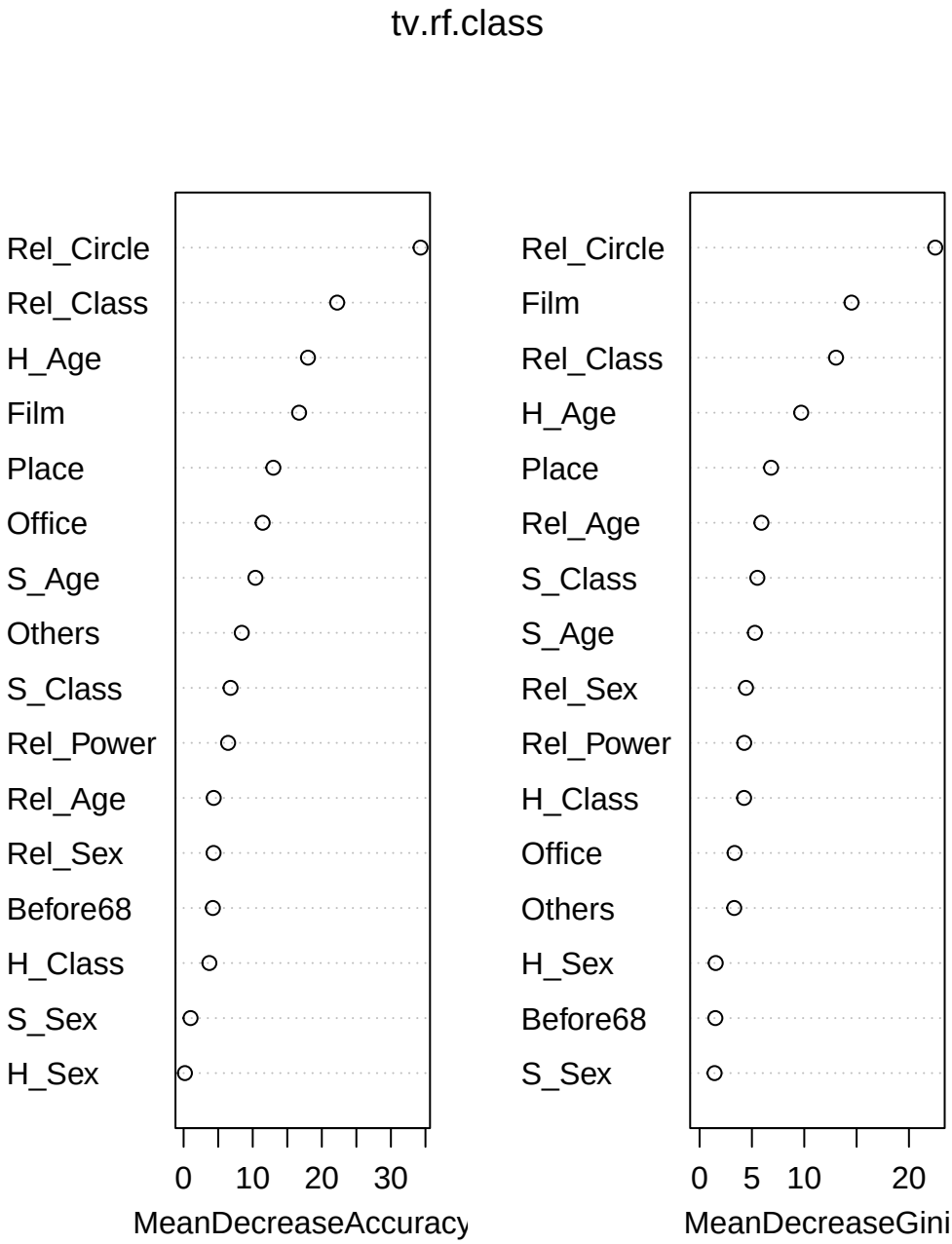
(`MeanDecreaseGini`) and the RSS for regression (`IncNodePurity`). However, these measures are **biased** towards predictors with many values, such as continuous variables or factors with many levels (cf. Strobl et al. 2007).

A more robust alternative is **permutation accuracy importance**, which compares the predictive accuracy of the random forest before and after randomly permuting the values of a predictor. If a predictor is important, permuting its values should lead to a substantial loss in accuracy. Since we have set `importance = TRUE`, `varImpPlot()` displays both measures side by side (`%IncMSE` and `MeanDecreaseAccuracy`, respectively, refer to permutation importance).

```
# Variable importance (Reaction times)
varImpPlot(rt.rf.reg)
```




```
# Variable importance (Form of 2.p.)
varImpPlot(tv.rf.class)
```



Note that the factor `Film`, which has many levels, may receive inflated Gini importance scores. Comparing its rank across both measures can help to detect this bias.

If predictors are correlated, standard permutation importance can still overestimate the importance of predictors that are merely correlated with truly important ones. **Conditional permutation accuracy importance** addresses this by permuting a predictor only within groups of observations with similar values on the correlated predictors (cf. Strobl et al. 2008; Debeer and Strobl 2020).

```
# Conditional permutation accuracy importance
library("permimp")

# Refit RF model with additional parameters
set.seed(123)

tv.rf.class <- randomForest(Form ~ .,
                            data = tv,
                            mtry = 4,
                            ntree = 500,
                            keep.inbag = TRUE,
                            keep.forest = TRUE)

# Compute CPI scores (do_check = FALSE skips the confirmation prompt in the
 ↪  console;
# make sure the data have not changed since fitting the model)
tv.rf.permimp <- permimp(tv.rf.class, conditional = TRUE, progressBar =
 ↪  FALSE, threshold = .95, do_check = FALSE)

# The same works for conditional random forests (and is considerably
 ↪  faster):
# tv.crf.permimp <- permimp(tv.crf.class, conditional = TRUE, progressBar =
 ↪  FALSE)

# Plot CPI scores
plot(tv.rf.permimp, horizontal = TRUE, type = "dot", sort = TRUE)
```

Visualising random forest models

Partial dependence plots show how the predictions of a model change with a given predictor X_s while averaging over the effects of all other predictors X_c. For each value x_s, the partial dependence function computes the prediction for every observation i in the training data, with X_s set to x_s and all other predictors left at their observed values x_{ic}, and averages these predictions:

$$\bar{f}_s(x_s) = \frac{1}{n}\sum_{i=1}^{n}\hat{f}(x_s, x_{ic}).$$

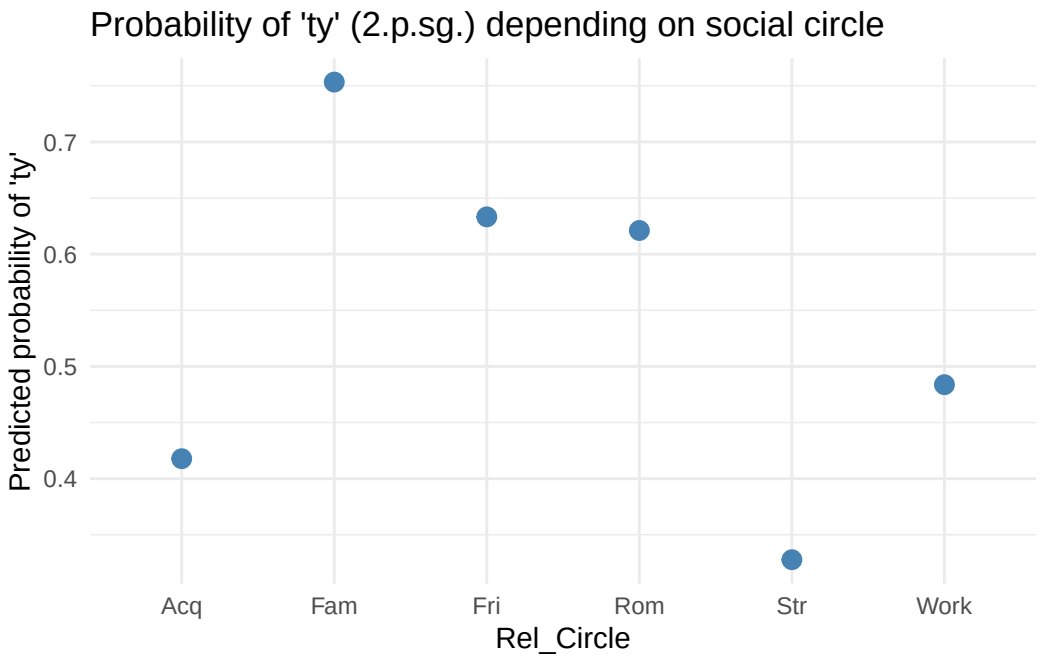
For a categorical response variable with K possible values, `pdp` uses the centred log-probability by default:

$$f_k(x) = \log[p_k(x)] - \frac{1}{K}\sum_{j=1}^{K}\log[p_j(x)],$$

with $p_k(x)$ corresponding to the fitted probability of the kth level of the response variable for $k \in \{1, 2, ..., K\}$ (B. M. Greenwell 2017: 430; see also Hastie, Tibshirani, and Friedman 2017). As this scale is hard to interpret (for a binary response, it amounts to half the log odds), we will request probabilities instead by setting `prob = TRUE`.

```
# Form ~ Rel_Circle
Rel_Circle.partial <- pdp::partial(tv.rf.class, pred.var = "Rel_Circle",
 ↪  which.class = "ty", prob = TRUE)

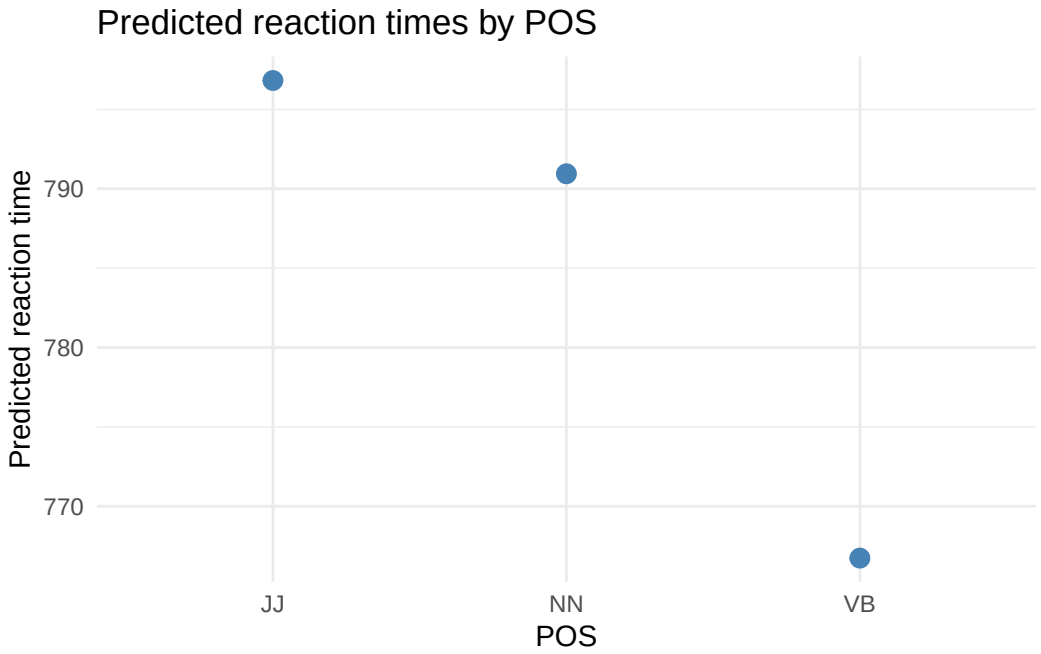
Rel_Circle.partial %>%
  ggplot(aes(x = Rel_Circle, y = yhat)) +
  geom_point(col = "steelblue", size = 3) +
  theme_minimal() +
  labs(
    title = "Probability of 'ty' (2.p.sg.) depending on social circle",
    y = "Predicted probability of 'ty'"
  )
```




```
# RT ~ POS
pos.partial <- pdp::partial(rt.rf.reg, pred.var = "POS")

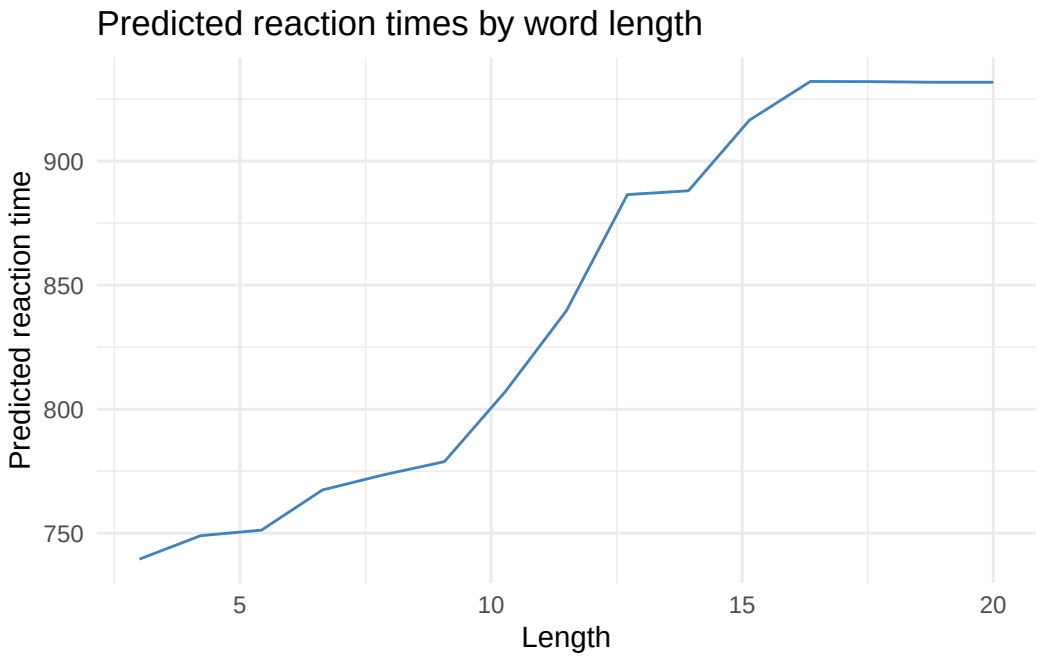
pos.partial %>%
  ggplot(aes(x = POS, y = yhat)) +
  geom_point(col = "steelblue", size = 3) +
  theme_minimal() +
  labs(
```

```
    title = "Predicted reaction times by POS",
    y = "Predicted reaction time"
  )
```




```
# RT ~ Length
length.partial <- pdp::partial(rt.rf.reg, pred.var = "Length")

length.partial %>%
  ggplot(aes(x = Length, y = yhat)) +
  geom_line(col = "steelblue") +
  theme_minimal() +
  labs(
    title = "Predicted reaction times by word length",
    y = "Predicted reaction time"
  )
```



Since `Rel_Circle` and `POS` are categorical, the predictions are shown as points rather than connected by lines, which would suggest a continuous scale.

References

Breiman, Leo. 1984. *Classification and Regression Trees*. Belmont, Calif.: Wadsworth International Group.

———. 2001. "Random Forests." *Machine Learning* 45 (1): 5–32. https://doi.org/10.1023/A:1010933404324.

Debeer, Dries, and Carolin Strobl. 2020. "Conditional Permutation Importance Revisited." *Bioinformatics* 21 (1): 307.

Greenwell, Brandon. 2022. *Tree-Based Methods for Statistical Learning in r*. London & New York: Taylor & Francis Group. https://search.ebscohost.com/login.aspx?direct=true&scope=site&db=nlebk&db=nlabk&AN=3288358.

Greenwell, Brandon M. 2017. "Pdp: An r Package for Constructing Partial Dependence Plots." *The R Journal* 9 (1): 421–36.

Gries, Stefan Thomas. 2021. *Statistics for Linguistics with r: A Practical Introduction*. 3rd rev. ed. Berlin; Boston: De Gruyter Mouton.

Hastie, Trevor, Robert Tibshirani, and Jerome H. Friedman. 2017. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2nd ed. New York, NY: Springer.

James, Gareth, Daniela Witten, Trevor Hastie, and Robert Tibshirani. 2021. *An Introduction to Statistical Learning: With Applications in r*. New York: Springer. https://doi.org/10.1007/978-1-0716-1418-1.

Levshina, Natalia. 2015. *How to Do Linguistics with r: Data Exploration and Statistical Analysis*. Amsterdam; Philadelphia: John Benjamins Publishing Company.

———. 2020. "Conditional Inference Trees and Random Forests." In *A Practical Handbook of Corpus Linguistics*, edited by Magali Paquot and Stefan Thomas Gries, 611–43. Cham: Springer.

Strobl, Carolin, Anne-Laure Boulesteix, Thomas Kneib, Thomas Augustin, and Achim Zeileis. 2008. "Conditional Variable Importance for Random Forests." *BMC Bioinformatics* 9 (1): 307.

Strobl, Carolin, Anne-Laure Boulesteix, Achim Zeileis, and Torsten Hothorn. 2007. "Bias in Random Forest Variable Importance Measures: Illustrations, Sources and a Solution." *BMC Bioinformatics* 8 (1): 25.

Strobl, Carolin, James Malley, and Gerhard Tutz. 2009. "An Introduction to Recursive Partitioning: Rationale, Application, and Characteristics of Classification and Regression Trees, Bagging, and Random Forests." *Psychological Methods* 14: 323–48. https://doi.org/10.1037/a0016973.